

Experiment – 3

Creating a Virtual Machine and Installing GCC/G++ Compiler

Aim: To create a new virtual machine using Oracle VM VirtualBox and install the build-essential package, which includes the GCC and G++ compilers on an Ubuntu Linux environment.

Prerequisites:

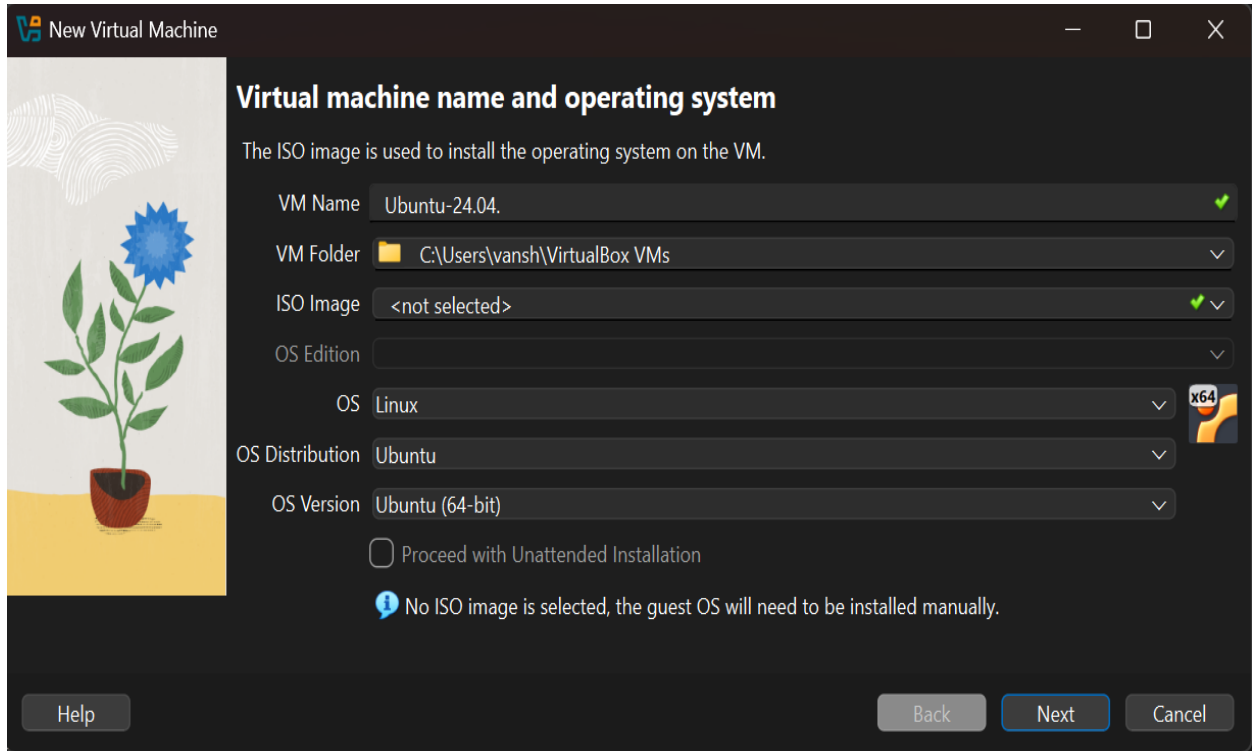
- **Hypervisor:** Oracle VM VirtualBox Manager.
- **Operating System Image:** Ubuntu 24.04 ISO file.
- **Host System:** A machine with sufficient RAM and CPU cores.

Theory:

- **Virtualization and Hypervisors:** Virtualization is the process of creating a software-based representation of something, such as applications, servers, storage, and networks. Oracle VM VirtualBox is a **Type-2 Hypervisor**, meaning it runs as an application on top of an existing operating system (the "Host") to manage multiple "Guest" operating systems.
- **Resource Allocation:** When creating a VM, the hypervisor carves out a specific portion of the host's physical hardware—such as RAM and CPU cycles—to be used exclusively by the Guest OS. This provides an isolated environment perfect for testing software without affecting the main system.
- **The Build-Essential Package:** In Linux distributions like Ubuntu, the build-essential package is a "meta-package". It doesn't contain software itself but acts as a list that triggers the installation of all tools necessary for compiling C/C++ code, including:
 - **GCC (GNU Compiler Collection):** The standard compiler for the C language.
 - **G++:** The compiler driver for C++.
 - **Make:** A utility that automates the process of building executable programs from source code.
 - **Standard C Libraries (libc-dev):** Headers and libraries required to interface with the Linux kernel

Procedure:

- **Launch VirtualBox:** Open Oracle VM VirtualBox and click on the "New" button to start the VM creation wizard.
- **Configure VM Name and OS:** * Set the **VM Name** to Ubuntu-24.04.
- Select the **Folder** where the VM will be stored.
- Set the **Type** to Linux and the **Version** to Ubuntu (64-bit).

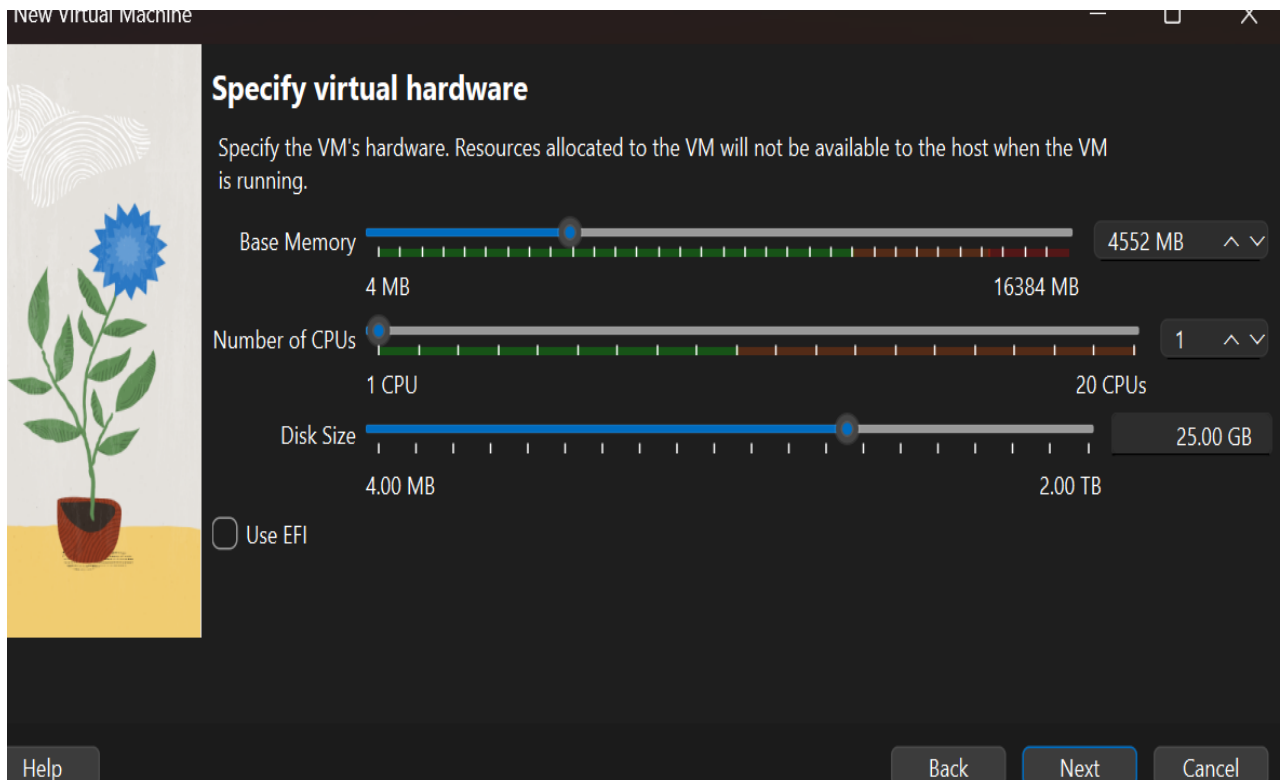


The screenshot shows the 'New Virtual Machine' wizard in Oracle VM VirtualBox. The title bar says 'New Virtual Machine'. The main heading is 'Virtual machine name and operating system'. Below it, a note states: 'The ISO image is used to install the operating system on the VM.' The configuration fields are as follows:

- VM Name: Ubuntu-24.04. (with a green checkmark icon)
- VM Folder: C:\Users\vansh\VirtualBox VMs (with a folder icon)
- ISO Image: <not selected> (with a green checkmark icon)
- OS Edition: (empty dropdown)
- OS: Linux (dropdown menu)
- OS Distribution: Ubuntu (dropdown menu)
- OS Version: Ubuntu (64-bit) (dropdown menu)

There is a checkbox for 'Proceed with Unattended Installation' which is unchecked. A warning icon (light blue circle with an exclamation mark) is present with the text: 'No ISO image is selected, the guest OS will need to be installed manually.' At the bottom, there are three buttons: 'Help', 'Back', and 'Next' (highlighted with a blue border), and a 'Cancel' button.

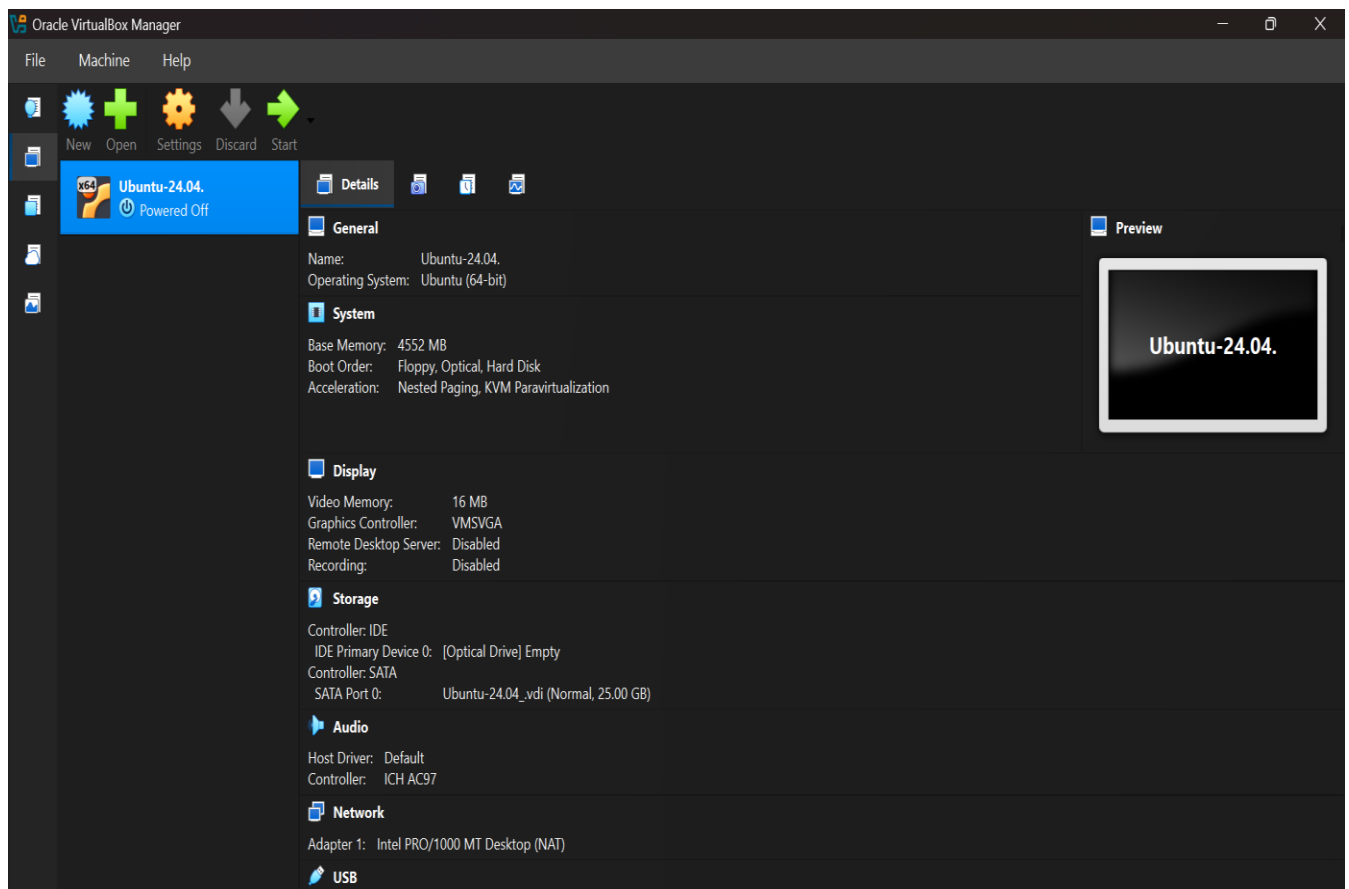
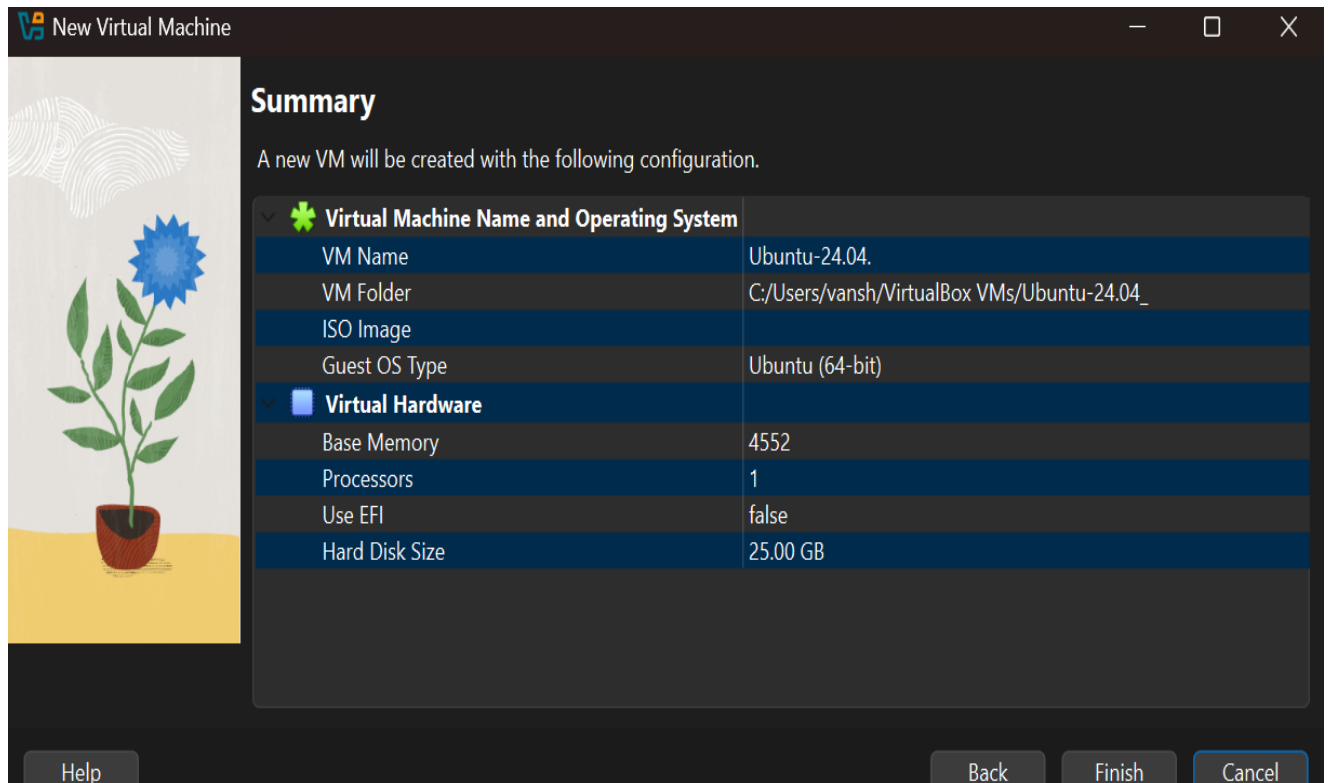
- **Specify Virtual Hardware:**
- **Base Memory:** Allocate at least **4552 MB** of RAM.
- **Processors:** Allocate **1 CPU** core.
- **Create Virtual Hard Disk:** Set the **Disk Size** to **25.00 GB**.



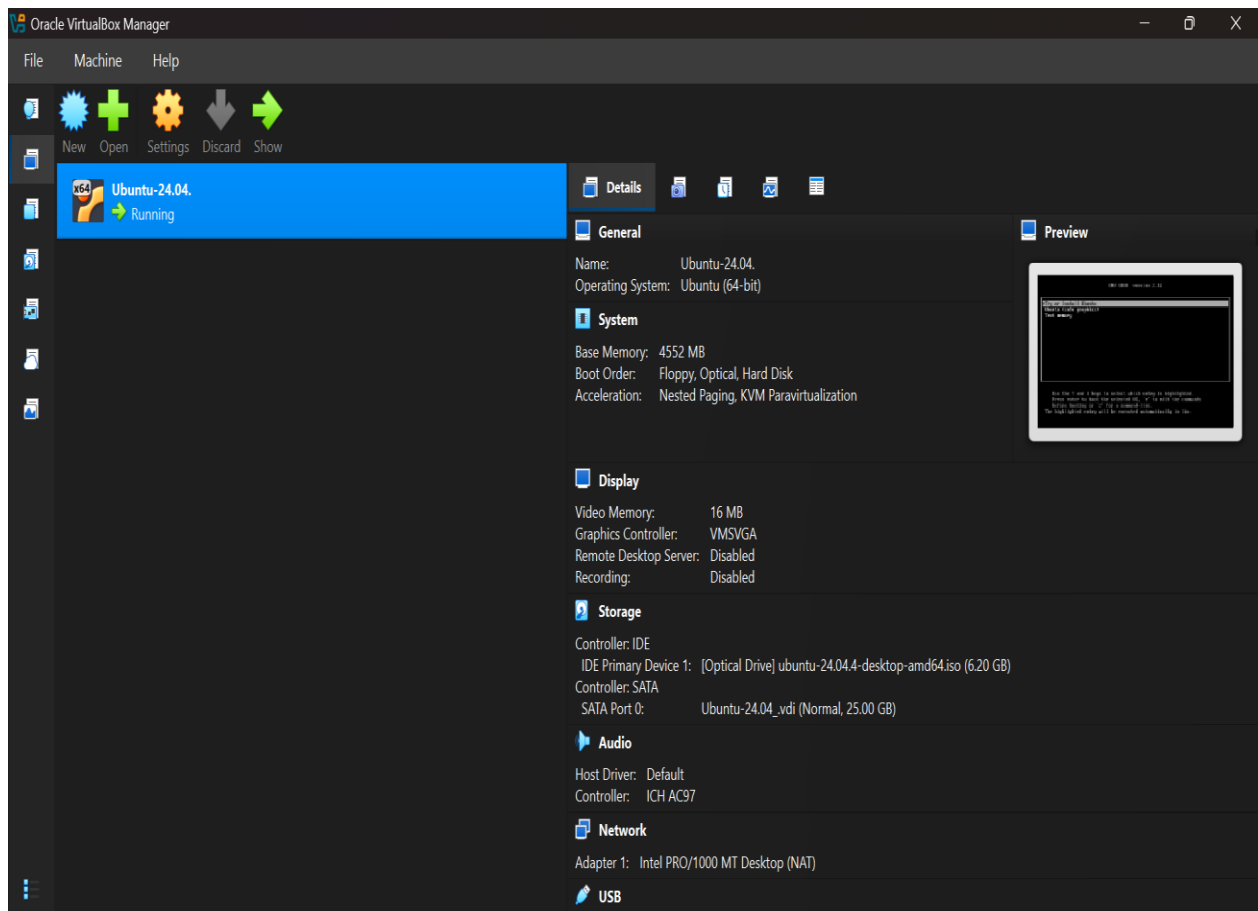
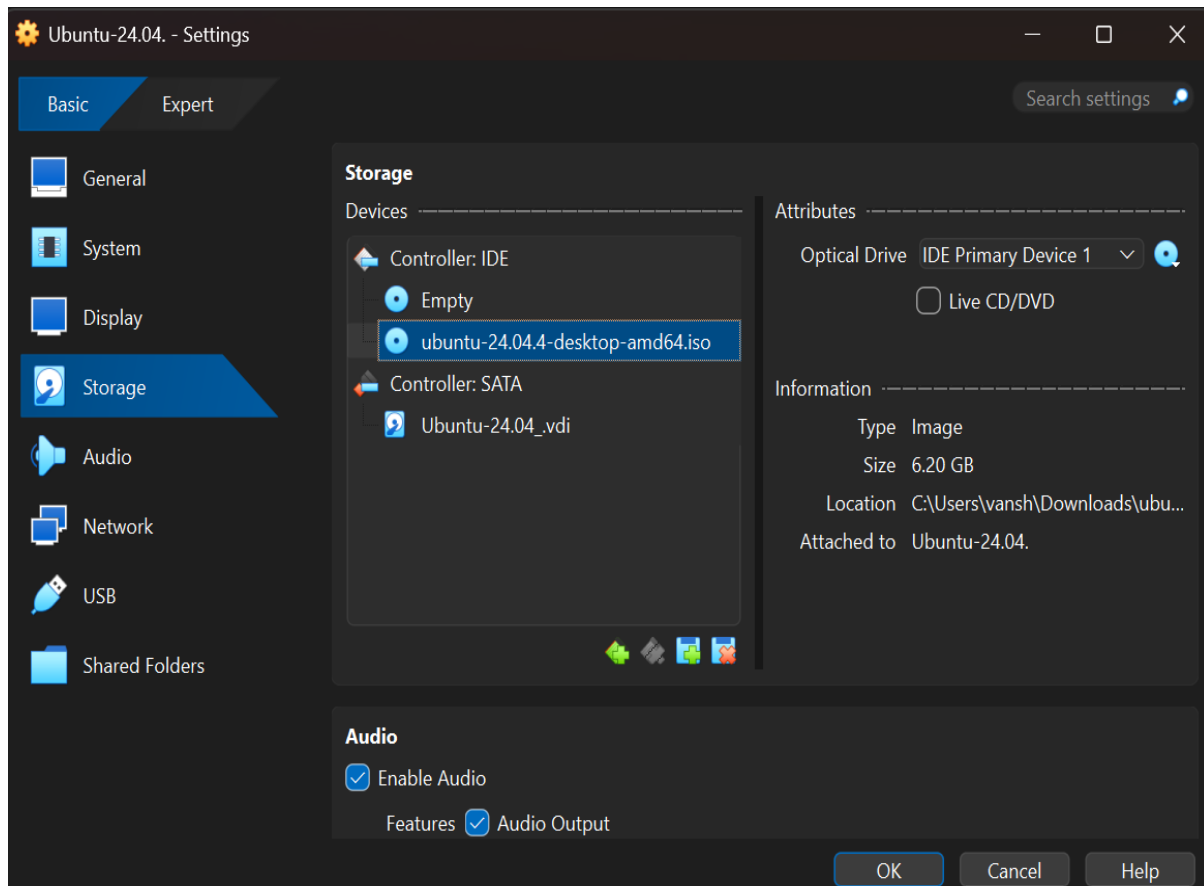
The screenshot shows the 'New Virtual Machine' wizard in Oracle VM VirtualBox, Step 2: 'Specify virtual hardware'. The title bar says 'New Virtual Machine'. The main heading is 'Specify virtual hardware'. Below it, a note states: 'Specify the VM's hardware. Resources allocated to the VM will not be available to the host when the VM is running.' The configuration fields are as follows:

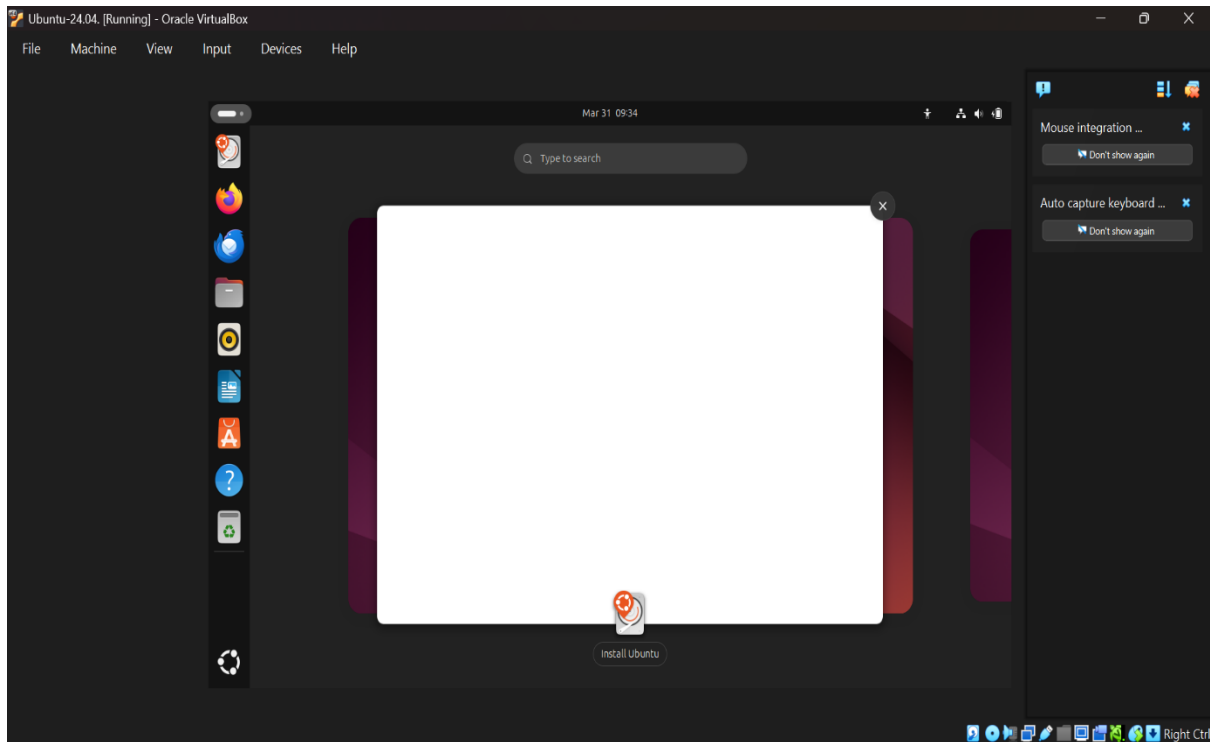
- Base Memory: A slider bar ranging from 4 MB to 16384 MB, with a value of 4552 MB displayed on the right.
- Number of CPUs: A slider bar ranging from 1 CPU to 20 CPUs, with a value of 1 displayed on the right.
- Disk Size: A slider bar ranging from 4.00 MB to 2.00 TB, with a value of 25.00 GB displayed on the right.

There is a checkbox for 'Use EFI' which is unchecked. At the bottom, there are three buttons: 'Help', 'Back', and 'Next' (highlighted with a blue border), and a 'Cancel' button.



- **Mount ISO and Start:** In the VM settings, go to **Storage**, select the empty optical drive, and choose the ubuntu-24.04.4-desktop-amd64.iso file. Start the VM and follow the on-screen Ubuntu installation steps.





- **Update Repository:** Open the terminal and run the command to refresh the package list:
`sudo apt update.`

```
ubuntu@ubuntu: ~  
ubuntu@ubuntu:~$ sudo apt update  
Ign:1 cdrom://Ubuntu 24.04.4 LTS _Noble Numbat_ - Release amd64 (20260210) noble  
InRelease  
Hit:2 cdrom://Ubuntu 24.04.4 LTS _Noble Numbat_ - Release amd64 (20260210) noble  
Release  
Hit:4 http://archive.ubuntu.com/ubuntu noble InRelease  
Get:5 http://archive.ubuntu.com/ubuntu noble-updates InRelease [126 kB]  
Get:6 http://security.ubuntu.com/ubuntu noble-security InRelease [126 kB]  
Get:7 http://archive.ubuntu.com/ubuntu noble-backports InRelease [126 kB]  
Get:8 http://security.ubuntu.com/ubuntu noble-security/main amd64 Packages [1551  
kB]  
Get:9 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 Packages [1847 k  
B]  
Get:10 http://archive.ubuntu.com/ubuntu noble-updates/main i386 Packages [599 kB  
]  
Get:11 http://security.ubuntu.com/ubuntu noble-security/main i386 Packages [396  
kB]  
Get:12 http://security.ubuntu.com/ubuntu noble-security/main Translation-en [250  
kB]  
Get:13 http://security.ubuntu.com/ubuntu noble-security/main amd64 Components [2  
1.5 kB]  
Get:14 http://security.ubuntu.com/ubuntu noble-security/main amd64 c-n-f Metadat  
a [10.7 kB]  
Get:15 http://security.ubuntu.com/ubuntu noble-security/universe i386 Packages [
```

- **Install Compilers:** Install the necessary compiler tools by running:
`sudo apt install build-essential.`

```
ubuntu@ubuntu: ~  
151 packages can be upgraded. Run 'apt list --upgradable' to see them.  
ubuntu@ubuntu:~$ sudo apt install build-essential  
Reading package lists... Done  
Building dependency tree... Done  
Reading state information... Done  
The following additional packages will be installed:  
  binutils binutils-common binutils-x86-64-linux-gnu cpp-13  
  cpp-13-x86-64-linux-gnu dpkg-dev fakeroot g++ g++-13 g++-13-x86-64-linux-gnu  
  g++-x86-64-linux-gnu gcc gcc-13 gcc-13-base gcc-13-x86-64-linux-gnu  
  gcc-14-base gcc-x86-64-linux-gnu libalgorithm-diff-perl  
  libalgorithm-diff-xs-perl libalgorithm-merge-perl libasan8 libatomic1  
  libbinutils libbcc1-0 libctf-nobfd0 libctf0 libdpkg-perl libfakeroot  
  libfile-fcntllock-perl libgcc-13-dev libgcc-s1 libgomp1 libgprofng0  
  libhwasan0 libitm1 liblsan0 libquadmath0 libsframe1 libstdc++-13-dev  
  libstdc++6 libtsan2 libubsan1 lto-disabled-list make  
Suggested packages:  
  binutils-doc gprofng-gui gcc-13-locales cpp-13-doc debian-keyring  
  g++-multilib g++-13-multilib gcc-13-doc gcc-multilib autoconf automake  
  libtool flex bison gcc-doc gcc-13-multilib gdb-x86-64-linux-gnu git bzip  
  libstdc++-13-doc make-doc  
The following NEW packages will be installed:  
  binutils binutils-common binutils-x86-64-linux-gnu build-essential dpkg-dev  
  fakeroot g++ g++-13 g++-13-x86-64-linux-gnu g++-x86-64-linux-gnu gcc gcc-13  
  gcc-13-x86-64-linux-gnu gcc-x86-64-linux-gnu libalgorithm-diff-perl
```

- **Verify Installation:** Check the installed GCC version:
`gcc --version`

```
Setting up gcc-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...  
Setting up gcc (4:13.2.0-7ubuntu1) ...  
Setting up g++-x86-64-linux-gnu (4:13.2.0-7ubuntu1) ...  
Setting up g++-13 (13.3.0-6ubuntu2~24.04.1) ...  
Setting up g++ (4:13.2.0-7ubuntu1) ...  
update-alternatives: using /usr/bin/g++ to provide /usr/bin/c++ (c++) in auto mo  
de  
Setting up build-essential (12.10ubuntu1) ...  
Processing triggers for man-db (2.12.0-4build2) ...  
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...  
ubuntu@ubuntu:~$ gcc --version  
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0  
Copyright (C) 2023 Free Software Foundation, Inc.  
This is free software; see the source for copying conditions. There is NO  
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.  
ubuntu@ubuntu:~$
```

Output:

- A successfully created and running Ubuntu 24.04 Virtual Machine.
- Terminal output showing: gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0.

Experiment – 4

Creating and Running a "Hello World" C Program

Aim: To write, compile, and execute a basic C program ("Hello World") within the newly created Ubuntu Virtual Machine environment.

Prerequisites:

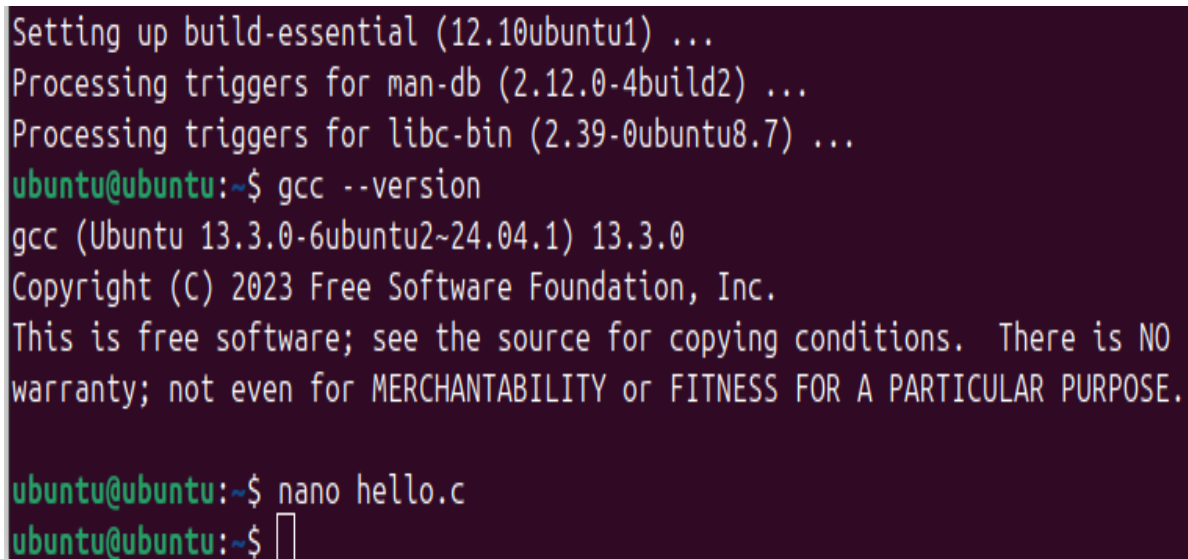
- Functional Ubuntu VM from Experiment 3.
- GCC compiler installed.
- Text editor (e.g., Nano).

Theory:

- **The Compilation Workflow:** Writing code is only the first step. The computer cannot understand C code directly; it must go through a transformation process:
 1. **Preprocessing:** Handling directives (like `#include`) to bring in necessary libraries.
 2. **Compilation:** Translating the C code into Assembly code.
 3. **Assembly:** Converting Assembly into Machine Code (object files).
 4. **Linking:** Combining the object code with system libraries to create a single executable file.
- **The GNU Nano Editor:** Nano is a simple, modeless command-line text editor for Unix-like systems. It is ideal for quick coding tasks directly within the terminal.
- **Standard I/O (`stdio.h`):** This header file stands for "Standard Input-Output." It provides the definitions for functions like `printf()`, which allows the program to send data to the "Standard Output" (the terminal screen).

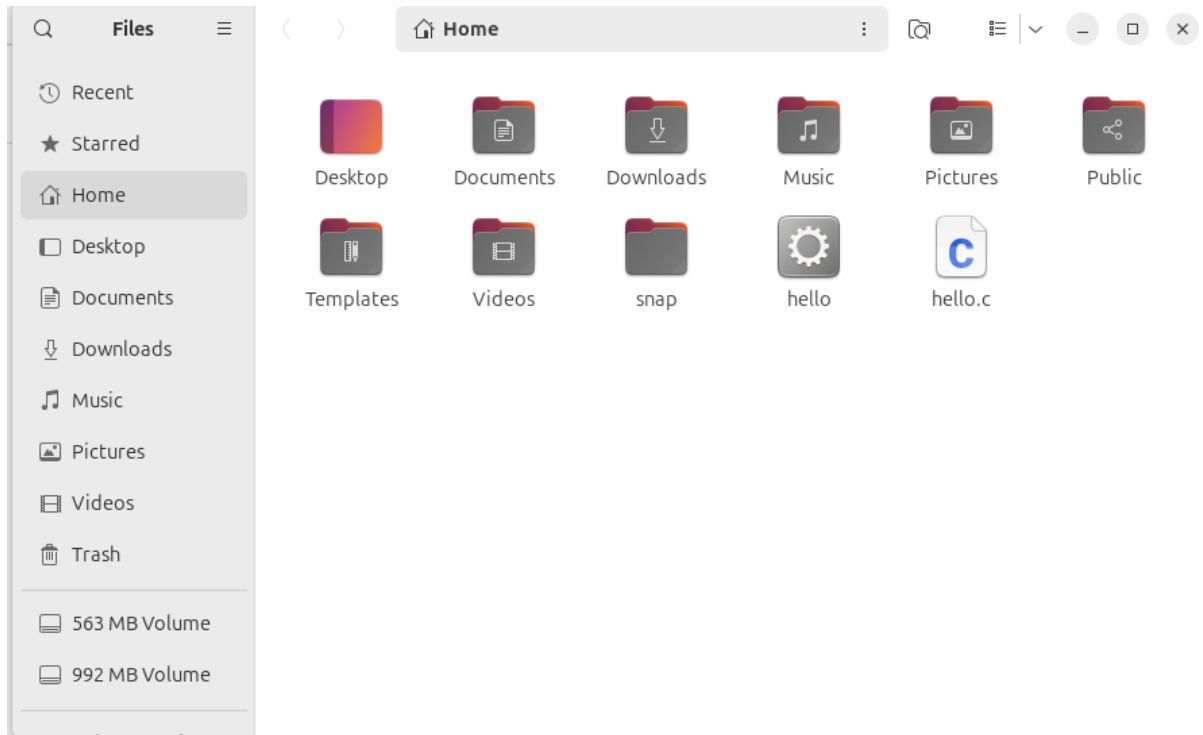
Procedure:

1. **Create Source File:** Open the terminal and use the nano editor to create a new file:
`nano hello.c.`



```
Setting up build-essential (12.10ubuntu1) ...
Processing triggers for man-db (2.12.0-4build2) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
ubuntu@ubuntu:~$ gcc --version
gcc (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
Copyright (C) 2023 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

ubuntu@ubuntu:~$ nano hello.c
ubuntu@ubuntu:~$
```



2. **Write the Code:** Type the C program into the editor.

A screenshot of a terminal window with the title 'ubuntu@ubuntu: ~'. The terminal shows the GNU nano 7.2 editor editing a file named 'hello.c'. The code in the editor is:

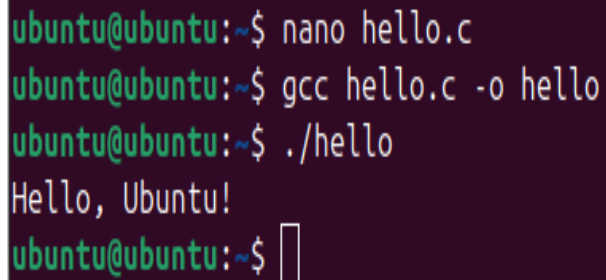
```
#include <stdio.h>
int main() {
    printf("Hello, Ubuntu!\n");
    return 0;
}
```

The bottom of the terminal window displays a status bar with various keyboard shortcuts: ^G Help, ^O Write Out, ^W Where Is, ^K Cut, ^T Execute, ^C Location, ^X Exit, ^R Read File, ^\ Replace, ^U Paste, ^J Justify, and ^_ Go To Line.

3. **Save and Exit:** Press Ctrl+O to write out the file and Ctrl+X to exit the editor.
4. **Compile the Code:** Use GCC to compile the source file into an executable named hello:

gcc hello.c -o hello.

5. **Execute the Program:** Run the compiled executable from the terminal:
./hello.



```
ubuntu@ubuntu:~$ nano hello.c
ubuntu@ubuntu:~$ gcc hello.c -o hello
ubuntu@ubuntu:~$ ./hello
Hello, Ubuntu!
ubuntu@ubuntu:~$
```

Source Code

```
#include <stdio.h> // Header file for standard input and output
int main() { // Main function where execution starts
    printf("Hello, Ubuntu!\n"); // Prints the string to the console
    return 0; // Indicates successful program termination
}
```

Output:

The terminal displays the message:
Hello, Ubuntu!.
